

Ejercicio 9: El Problema de la Mochila 0-1 Especial (Algoritmo Voraz)

Este documento presenta la formalización, el diseño de un algoritmo voraz eficiente y la demostración formal simplificada de su corrección para una variante especial del problema de la mochila 0-1 donde el orden de los objetos por pesos crecientes coincide con el de beneficios decrecientes.

1. Análisis del Problema y Propiedad de Dominancia

Disponemos de una mochila de capacidad máxima W y de un conjunto de n objetos indivisibles. Cada objeto i tiene un peso $w_i > 0$ y un beneficio $v_i > 0$.

Condición Especial del Problema

El enunciado establece que si ordenamos los objetos por peso creciente, obtenemos el mismo orden que si los ordenamos por beneficio decreciente. Si reindexamos los objetos según este orden, se cumple estrictamente que:

$$w_1 \leq w_2 \leq \dots \leq w_n \quad \text{y} \quad v_1 \geq v_2 \geq \dots \geq v_n$$

Propiedad de Dominancia Absoluta

De esta relación se deduce que los objetos con menor índice dominan absolutamente a los objetos con mayor índice en las dos dimensiones del problema:

- Tienen un peso menor o igual: Requieren menos capacidad de la mochila ($w_i \leq w_j$ para todo $i < j$).
- Tienen un beneficio mayor o igual: Aportan más valor a la solución ($v_i \geq v_j$ para todo $i < j$).

Por tanto, ante la decisión de incluir un objeto en la mochila, siempre será preferible (o al menos igual de óptimo) elegir el objeto i antes que cualquier objeto posterior $j > i$.

2. Estrategia Algorítmica (Elección Voraz)

Dada la propiedad de dominancia absoluta, la estrategia voraz óptima consiste en **seleccionar los objetos estrictamente en el orden de sus índices** (de menor a mayor peso / de mayor a menor beneficio) hasta que el siguiente objeto de la lista no quepa en la capacidad restante de la mochila.

- **Paso Voraz:** En la etapa i , si el objeto i cabe en el espacio restante de la mochila ($w_i \leq W_{\text{restante}}$), se incluye obligatoriamente en la solución ($x_i = 1$). El algoritmo finaliza en el primer elemento k que no quepa en la mochila.

Dado que cualquier objeto posterior $j > k$ tendrá un peso $w_j \geq w_k$, y dado que w_k ya excede la capacidad restante, podemos asegurar que **ningún objeto posterior a k cabrá en la mochila**.

Por tanto, la solución voraz es un prefijo de la secuencia ordenada:

$$X = (\underbrace{1, 1, \dots, 1}_{k \text{ elementos}}, 0, \dots, 0)$$

3. Algoritmo en Pseudocódigo

A continuación se presenta la implementación del algoritmo utilizando índices base 1:

```

Funcion ResolverMochilaEspecial(W, w, v, n)
Inicio
    // 1. Ordenar los elementos. Como el orden de pesos crecientes es el mismo
    // que el de beneficios decrecientes, ordenamos por peso creciente en O(n log
    // Conservamos los índices emparejados de pesos y beneficios.
    (w_ord, v_ord) <- OrdenarPorPesoCreciente(w, v, n)

    x <- Vector de tamaño n inicializado en 0 // Vector solución
    capacidad_restante <- W
    Z_beneficio_total <- 0

    // 2. Selección Voraz
    Para i <- 1 Hasta n Con Paso 1 Hacer
        Si w_ord[i] <= capacidad_restante Entonces
            x[i] <- 1
            capacidad_restante <- capacidad_restante - w_ord[i]
            Z_beneficio_total <- Z_beneficio_total + v_ord[i]
        Sino
            // Como w_ord está ordenado ascendentemente, si el elemento i no cabe
            // ningún elemento posterior j > i cabrá (ya que w_ord[j] >= w_ord[i])
            Romper Bucle
    FinSi
FinPara

    Escribir("Beneficio óptimo de la mochila: ", Z_beneficio_total)
    Retornar x
FinFuncion

```

4. Análisis de Coste

- **Complejidad Temporal:** $\Theta(n \log n)$ en el caso general. El paso dominante es la ordenación de los vectores de entrada mediante un algoritmo eficiente como *MergeSort*, que requiere un coste de $\Theta(n \log n)$. El bucle de asignación voraz realiza a lo sumo n iteraciones de coste constante, aportando un coste lineal de $O(n)$. (Nota: Si los datos de entrada ya se proporcionan ordenados de fábrica, el coste del algoritmo es *strictly lineal*: $\Theta(n)$).
- **Complejidad Espacial:** $O(n)$ de memoria adicional para almacenar la copia ordenada de los objetos y el vector de asignación final x .

5. Justificación de la Corrección (Demostración de Optimalidad)

Para demostrar que la solución voraz V es óptima, usamos un **argumento de intercambio** simplificado.

Sean:

- $V = (v_1, \dots, v_n)$ la solución voraz (un prefijo de unos: $1, \dots, 1, 0, \dots, 0$).
- $O = (o_1, \dots, o_n)$ una solución óptima cualquiera tal que $O \neq V$.

Demostración:

1. Sea i el primer índice donde difieren las soluciones ($o_i \neq v_i$). Necesariamente se cumple que $o_i = 0$ y $v_i = 1$ (si el voraz puso $v_i = 0$ es porque el objeto no cabía, luego en O tampoco cabría).
2. Como O es óptima, debe haber seleccionado algún objeto posterior $j > i$ que sí está en la mochila ($o_j = 1$).
3. Realizamos un intercambio en O : quitamos el objeto j e introducimos el objeto i en su lugar, obteniendo una nueva solución O' .
4. Evaluamos la validez y el beneficio de O' :
 - **Viabilidad física:** Como $i < j$, se cumple que $w_i \leq w_j$. El peso total no aumenta, por lo que la mochila sigue siendo factible.
 - **Beneficio obtenido:** Como $i < j$, se cumple que $v_i \geq v_j$. El beneficio de la nueva solución no disminuye:

$$\Delta Z = v_i - v_j \geq 0$$

5. Repitiendo este intercambio para todas las diferencias, transformamos la solución óptima O en la solución voraz V sin empeorar en ningún paso el beneficio total.

Por tanto, la solución voraz V es globalmente óptima.